Lab 7: Linux Capabilities and Least Privilege

Computing Systems Security, 2026-27 MCS and MEI, University of Minho

Vítor Francisco Fonte vff@di.uminho.pt

Lab 6 showed privilege as all-or-nothing: a process is root or it is not, and setuid grants the whole of root when perhaps a sliver was needed. Linux **capabilities** decompose root into many independent rights, so a process can be granted exactly the one privilege it requires and nothing more. This lab has you enumerate the capabilities active on a running system and inside containers, attach a **file capability** to a binary instead of making it setuid root (the fix you reached for in Lab 6), and apply least privilege to a real service. The point throughout: **splitting root is the right idea, and it is harder to do correctly than it looks.**

Safety. No self-lockout risk here, but you will attach and remove file capabilities on binaries. Snapshot `lab7-clean` before you start. Pull the container images in the pre-lab. Exercises 3 and 5 also run `apk add`, so the session needs network access.

Contents

- Learning Objectives
- Before the lab (at home, ~30-45 min)
- Part A: Capabilities on the running host
- Part B: File capabilities instead of setuid
- Part C: Capabilities inside containers
- Part D: Least privilege for a real service
- Part E: A non-root service with one capability
- If it does not work
- Reflection
- References
- Deliverables

Learning Objectives

By completing this lab, students will be able to:

1. Name the capability sets (permitted, effective, inheritable, ambient, bounding) and read a process's `CapEff` from `/proc`.
2. Enumerate the capabilities a process or container holds and decode the bitmask.
3. Attach a **file capability** to a binary and explain why it is a smaller grant than setuid root.
4. Explain why `--privileged` under rootful Docker is effectively host root, and why "runs as non-root" is not "is sandboxed" (a non-root UID alone does not prevent acquiring a capability). *(The full "capabilities alone are not containment" argument needs namespaces and seccomp and is completed in Lab 12; here you only reach the "non-root is not sandboxed" observation of Exercise 3b.)*
5. Apply least privilege to a service by granting the single capability it needs.

Capability enumeration and dropping are woven into the capstone's container-reaching link.

What from this lab is assessed. Two things, both handed in with **Lab report 3** at the end of the block (Lab 8): your answers to the preparation questions **P1 and P2**, and your **Reflection**. The exercises themselves are not handed in. Keep your capability transcripts: Lab 11 and the capstone reuse the least-privilege reasoning you build here.

Before the lab (at home, ~30-45 min)

Boot the VM, snapshot `lab7-clean`, re-read Lecture 7 if you have already had it. Pull the images the container exercises use so the session is not spent waiting:

```
docker pull alpine
docker pull python:3.12-alpine
```

Unzip `lab7-files.zip` into `~/ssc-lab7` and run every command of this lab from there (it contains `lab7-bind80.c`, the bind helper you compile in Part B; `captest.py`, a small bind-a-socket probe; and `Docker-file.capbind`, the image you build in Exercise 5). Confirm `capsh` and `getcap`/`setcap` are present:

```
which capsh getcap setcap        # from libcap; installed on the image
```

Preparation questions (submit with Lab report 3)

- **P1.** Roughly how many capabilities does Linux split root into, and name two of them with what each grants.
- **P2.** In one sentence, why is a file capability on a binary a smaller grant than the setuid-root bit?

Part A: Capabilities on the running host

Exercise 1. Read the capability sets of your own shell, then of a privileged process. These commands are probably new to you, so first what you are looking at:

- **`/proc/self/status`** is the kernel's report on the process that reads it (here `grep`, whose sets match your shell's in this exercise). `grep Cap` keeps five lines, one per capability set.
- **Each value is a bitmask in hexadecimal**, one bit per capability. `0000000000000000` is an empty set; `000001ffffffffff` has the lowest 41 bits on, which is every capability on this kernel.
- **`capsh --decode=<mask>`** turns a mask into capability names.

Start with your own shell, decoding two of its sets:

```
grep Cap /proc/self/status
capsh --decode=$(grep CapEff /proc/self/status | awk '{print $2}')
capsh --decode=$(grep CapBnd /proc/self/status | awk '{print $2}')
```

Now a system daemon running as root:

```
pid=$(pgrep -n cron || pgrep -n systemd)
sudo grep Cap /proc/$pid/status
capsh --decode=$(sudo grep CapEff /proc/$pid/status | awk '{print $2}')
```

1a. Compare your shell's `CapEff` with the daemon's. Name the five capability sets and state, in one sentence each, what `CapEff` (effective) and `CapBnd` (bounding) control. Why does the *effective* set, not the *permitted* set, determine what a syscall is allowed to do right now?

Part B: File capabilities instead of setuid

In Lab 6 you tried, among other options, replacing the setuid `backup-tool`'s blanket root with the specific capability it needs, and measured that it narrowed the grant without closing the hole. Here you apply the same mechanism to a program whose one need it does fit, and see the difference.

Exercise 2. Take a program that needs one privilege and grant it that privilege two ways: setuid root, then a file capability. Compare.

`lab7-bind80.c`, from `lab7-files.zip`, tries to bind TCP port 80 and prints the result with its `uid` and `euid`. On this VM ports below 1024 are privileged, so an ordinary process should be refused. Compile it, then run it:

```
gcc -o ~/capbind lab7-bind80.c
~/capbind
```

The compile prints nothing; the message comes from running the program.

The blunt way: setuid root (delete this copy when you finish the exercise). Make a copy, give it to root and set the setuid bit:

```
sudo cp ~/capbind ~/capbind_suid
sudo chown root ~/capbind_suid
sudo chmod u+s ~/capbind_suid
```

Try to run the program again:

```
~/capbind_suid
```

The sharp way: one file capability, no setuid. `cap_net_bind_service` is the capability that allows binding a privileged port. In `+ep`, the `p` asks for it in the file's permitted set and the `e` turns on the file's effective bit, a single flag that makes the capability active as soon as the program starts. Exercise 6 comes back to that flag.

```
sudo cp ~/capbind ~/capbind_cap
sudo setcap cap_net_bind_service+ep ~/capbind_cap
getcap ~/capbind_cap
```

Run this copy:

```
~/capbind_cap
```

Read `uid` and `euid` in all three runs. The contrast the exercise wants is between three runs of one program. Setting or removing a file capability needs `sudo`.

2a. Both let the binary bind port 80. Explain the security difference precisely. Under the setuid version, what is the process's effective UID while it runs, and everything that UID can do? Under the file-capability version, what extra privilege does the process have beyond binding a privileged port? This is the least-privilege argument made concrete: name what the second option *denied itself*.

2b. Remove the capability from `~/capbind_cap` and confirm the bind now fails. The command is not given here: find it in `setcap(8)`. Which command did you use, and why is it the correct clean-up? Then look at a program on this image that faced the same choice: `ping` classically opens a raw socket, a privileged operation, and for years that meant installing it setuid root. What does `getcap` show on this image's `ping`, and why does `ping` here not need the setuid-root bit?

Part C: Capabilities inside containers

The arc of Parts C, D and E. One question in three steps: *how much of root is a container's root* (Part C), *how small can a service's grant be* (Part D), and *how do you run that service as non-root with exactly the one capability it needs* (Part E). Each step corrects the one before, so keep the destination in view: Exercise 3 reads the default and privileged sets; 3b adds a non-root UID; Exercise 4 looks for the minimal grant; Exercise 5 makes it genuinely non-root; and Exercise 6 matches the ceiling (`CapBnd`) to the grant and shows what breaks when it drops too far.

Exercise 3. Compare the default and privileged container capability sets.

A process in a container is an ordinary process on the host's kernel, and capabilities apply to it as they do to any other. So the question that matters is what a container's root can do to the host. Docker starts containers with a reduced set; the `--privileged` flag asks for no reduction. You read the five sets in each case and decode `CapEff` to see how much of root a default container keeps and what `--privileged` changes.

The default container:

```
docker run --rm alpine sh -c "grep Cap /proc/self/status"
docker run --rm alpine sh -c \
  "apk add -q libcap && capsh --decode=\$(grep CapEff /proc/self/status | awk
   ↪  '{print \$2}')"
```

The privileged container:

```
docker run --rm --privileged alpine sh -c "grep Cap /proc/self/status"
docker run --rm --privileged alpine sh -c \
  "apk add -q libcap && capsh --decode=\$(grep CapEff /proc/self/status | awk
   ↪  '{print \$2}')"
```

3a. Record both runs; the privileged mask is one you have already seen in Exercise 1. Docker's default grants roughly 14 capabilities; `--privileged` grants all of them. Explain why, under this image's rootful Docker, `--privileged` is effectively **host root**. Capabilities are part of the answer; it also opens host devices and lifts other restrictions. Name two capabilities it adds and what each opens. Under what mistaken belief might an operator run `--privileged` thinking it safe?

3b. Does a non-root UID contain a container? **Measure first, and the measurement is the point**: run `--user 1000` in a default container and read `CapPrm`, `CapEff` and `CapBnd`. What does the process hold, compared with the default container's root run above, and what is left in `CapBnd`, the set this lab calls the ceiling: a mask the kernel applies at `execve` to the file's permitted capabilities? It does not take away capabilities a process already holds.

So what the process holds now is not the whole question. Think back to Part B: name a route by which a process in that container could still come to hold one of the default capabilities, say what must exist in the image for that route to work, name one of the capabilities and what it would open, and say what that means for treating ”runs as non-root” as if it were ”is sandboxed”.

Part D: Least privilege for a real service

Exercise 4. Find the smallest grant that lets a service bind a privileged port, and prove that anything less fails.

The service is `captest.py`, from `lab7-files.zip`: it tries to bind the port you give it and prints the result with its `uid`. Read the file before you run it. You run it in containers with Docker's `--cap-add` and `--cap-drop`. In every test, read the UID as well as the result, because the goal is a **non-root** service.

The kernel setting `net.ipv4.ip_unprivileged_port_start` is the lowest port an unprivileged process may bind. Its classical value is 1024, the privileged-port boundary, and below it the capability is what decides. Docker changes this value inside containers (4b asks how and why), so every test passes a flag that puts the classical value back. Keep the flag in a variable. It lives only in this shell: in a new terminal, set it again, or `$S` is empty and a bind can succeed for the wrong reason.

```
S="--sysctl net.ipv4.ip_unprivileged_port_start=1024"
```

Test 1: root, with Docker's default capabilities.

```
docker run --rm -v "$PWD/captest.py:/captest.py" $S python:3.12-alpine python
 ↪  /captest.py 80
```

Test 2: non-root, with no extra grant.

```
docker run --rm -v "$PWD/captest.py:/captest.py" $S --user 1000 python:3.12-alpine
 ↪  python /captest.py 80
```

Test 3: drop every capability and add back exactly `NET_BIND_SERVICE`.

```
docker run --rm -v "$PWD/captest.py:/captest.py" $S --cap-drop ALL --cap-add
 ↪  NET_BIND_SERVICE python:3.12-alpine python /captest.py 80
```

Test 4: the negative control the word ”minimal” needs. Drop every capability and add nothing back.

```
docker run --rm -v "$PWD/captest.py:/captest.py" $S --cap-drop ALL
 ↪  python:3.12-alpine python /captest.py 80
```

Tests 3 and 4 differ by one capability. That pair is what turns ”minimal” from a claim into a demonstration: without test 4 you have shown that a configuration works, not that the smaller one does not.

The result is not the whole story, so look at what the flags did to the sets. First with no flags, then with the flags of test 3:

```
docker run --rm python:3.12-alpine python -c
 ↪  "print(open('/proc/self/status').read())" | grep Cap
docker run --rm --cap-drop ALL --cap-add NET_BIND_SERVICE python:3.12-alpine python
 ↪  -c "print(open('/proc/self/status').read())" | grep Cap
```

4a. Answer in two steps.

- Record the outcomes and UIDs of tests 1 to 4. Among these tests, state the smallest grant that binds in one line (which capabilities dropped, which single one added), say what those flags did to `CapBnd`, and say whether it meets the non-root goal.
- Starting from Docker's default set, is `--cap-drop ALL --cap-add NET_BIND_SERVICE` a tighter or looser grant than the default? Why is ”start from nothing and add” the safer construction than ”start from default and remove”?

4b. Your tests set `ip_unprivileged_port_start` to 1024; Docker's own default inside a container is `0`. Run test 2 again without the flag:

```
docker run --rm -v "$PWD/captest.py:/captest.py" --user 1000 python:3.12-alpine
 ↪  python /captest.py 80
```

What does a value of `0` allow a process to do, and what practical benefit do you think Docker gets from it? In a container started without the flag, does `CAP_NET_BIND_SERVICE` decide whether a process can bind port 80? What does this tell you about reasoning from capabilities alone, without checking the surrounding configuration?

Part E: A non-root service with one capability

Look again at the UID in test 3. It binds as `uid=0`: root inside the container, holding one capability instead of the default fourteen. That is a real reduction, and it is not what Exercise 4 set out to find: it asked for a *non-root* service, and none of the four tests gives one that binds.

Exercise 5: make it actually non-root. The answer is not a Docker flag, it is the mechanism you already used in Part B, moved inside the image. `Dockerfile.capbind`, in `lab7-files.zip`, is `python:3.12-alpine` with one line added: `setcap cap_net_bind_service+ep` on the interpreter. Build it:

```
docker build -t lab7-capbind -f Dockerfile.capbind .
```

Run the service from your image as an ordinary user, with no `--cap-add`, then ask the interpreter what it is holding:

```
docker run --rm -v "$PWD/captest.py:/captest.py" $S --user 1000 lab7-capbind python
 ↪  /captest.py 80
docker run --rm --user 1000 lab7-capbind python -c
 ↪  "print(open('/proc/self/status').read())" | grep Cap
```

Now the control, and note that the image changes: the same two commands on the unmodified `python:3.12-alpine`, which has no file capability.

```
docker run --rm -v "$PWD/captest.py:/captest.py" $S --user 1000 python:3.12-alpine
 ↪  python /captest.py 80
docker run --rm --user 1000 python:3.12-alpine python -c
 ↪  "print(open('/proc/self/status').read())" | grep Cap
```

5a. Record the UID and the capability sets of both images. The four that carry the answer are `CapPrm`, `CapEff`, `CapAmb` and `CapBnd`; `CapInh` prints too and is zero throughout. What is the value of `CapPrm` in your image, and which capability does that bit stand for? Did it reach the process through the ambient set? What do you see in `CapBnd` in each image? Does the grant apply only to `captest.py`, or to anything else this interpreter runs?

Exercise 6: the grant is minimal, the ceiling is not. `CapBnd` in Exercise 5 still carries Docker's default set. In Exercise 4 you saw which flags bring it down, so use them here: non-root, on your image, with the ceiling reduced to the one capability. Bind, then read the sets:

```
docker run --rm -v "$PWD/captest.py:/captest.py" $S --user 1000 --cap-drop ALL
 ↪  --cap-add NET_BIND_SERVICE lab7-capbind python /captest.py 80
docker run --rm --user 1000 --cap-drop ALL --cap-add NET_BIND_SERVICE lab7-capbind
 ↪  python -c "print(open('/proc/self/status').read())" | grep Cap
```

Now take one step further and drop the ceiling below what the file capability needs:

```
docker run --rm -v "$PWD/captest.py:/captest.py" $S --user 1000 --cap-drop ALL
 ↪  lab7-capbind python /captest.py 80
```

6a. Record the UID and the capability sets of the service. Then look at the last command's output. What did you expect, and what happened? Explain it using two facts about the file capability and two sections of `capabilities(7)`:

- **+p** asks for the capability in the file's permitted set. Read ”Transformation of capabilities during execve()”.
- **+e** is the file's effective bit, one flag rather than a set of capabilities. Read ”Safety checking for capability-dumb binaries”.

Your answer needs both sections. Finish by saying what would change if the capability had been attached with +p alone.

6b. What is the minimal grant that lets a non-root service bind a privileged port, and why, on this image and Docker, can no combination of `--cap-add` and `--cap-drop` alone produce it? Tie your answer to what you did in Part B.

Conclusion

You might expect a non-root process on the unmodified `python:3.12-alpine` image, with `--cap-add NET_BIND_SERVICE` and no file capability, to bind port 80 under `$S` as well. It does **not**. These are the sets measured on the course image:

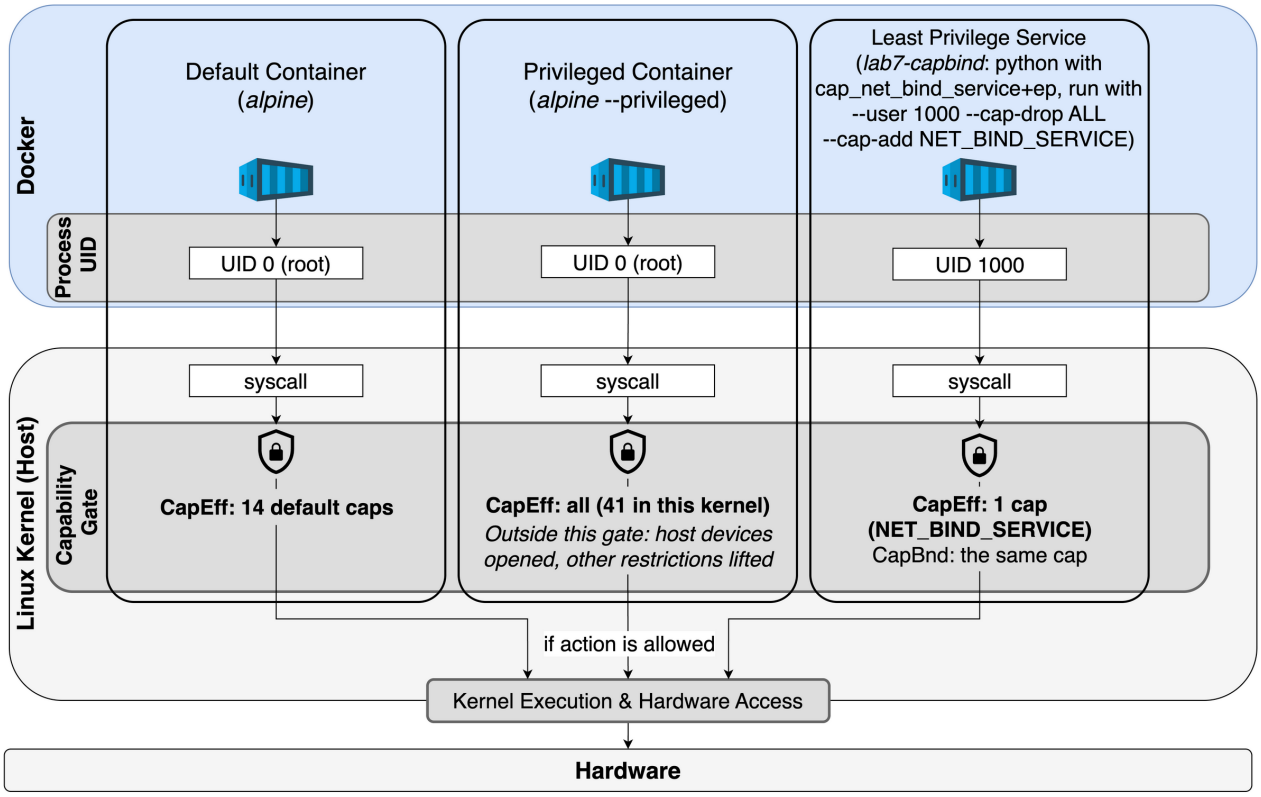
run on `python:3.12-alpine`	`CapPrm`	`CapEff`	`CapBnd`
`--cap-drop ALL --cap-add NET_BIND_SERVICE`, root inside	`0x400`	`0x400`	`0x400`
the same, plus `--user 1000`	`0`	`0`	`0x400`

These are two end states, not a sequence: they do not show the order in which the runtime built the sets, and another runtime may build them differently. In the second run the capability is in the bounding set but not in permitted or effective. Only the effective set is checked at the bind, so under `$S` the bind is refused. Without `--cap-drop ALL`, `--cap-add` changes nothing at all: `NET_BIND_SERVICE` is already in Docker's default bounding set, and `CapPrm` and `CapEff` stay at zero.

So with flags alone, the smallest grant is one capability held by root inside the container. A non-root service on this image needs a file capability on the binary (Parts B and E) or Docker's default open ports (4b).

The figure puts three of the cases you measured side by side: the default and privileged containers of Exercise 3, and the service of Exercise 6. In that third case the one capability comes from the file capability on the interpreter; the `--cap-drop` and `--cap-add` flags only lower the ceiling to match it. In all three,

when the process asks for a privileged operation, the kernel checks the `CapEff` it holds at the same gate. `--privileged` also changes things outside that gate: it opens host devices and lifts other restrictions (3a).



Copy your capability transcripts off the VM, then restore `lab7-clean`.

If it does not work

Both of these look like the capability did nothing. Neither is a broken lab.

What you see	What to do
`--user 1000 --cap-add NET_BIND_SERVICE` still cannot bind port 80 under `$S`	Measure it rather than assume: that run ends with `CapPrm = CapEff = 0` and `CapBnd` at Docker's default set, so the capability is in the bounding set and nowhere the process can use. `--cap-add` is a no-op here: `NET_BIND_SERVICE` is already in that default set. Use `--cap-drop ALL --cap-add NET_BIND_SERVICE` as root inside, or give the binary a **file capability** (Exercise 5).
The privileged-port boundary seems not to exist in a container	It does not, by default: `net.ipv4.ip_unprivileged_port_start` is 0 inside. Pass `--sysctl net.ipv4.ip_unprivileged_port_start=1024` to restore the boundary you are trying to observe.

If none of these matches, read the error message in full first: it usually names the layer that refused, and that is most of the answer. Then ask your lab teacher, in the session or by email.

Reflection

Your reflection goes into Lab report 3, as an appendix, beside the reflections from the other labs in this block. They are assessed with the report: every one has to be there, each answering its questions with a position you argue from what you did in the lab.

In a short piece (200-300 words):

1. Capabilities exist to make least privilege *possible*. Lab 6's setuid binary and this lab's file-capability version solve the same need with very different blast radii. Why, then, is setuid still everywhere? What makes capabilities harder to adopt in practice than the idea suggests?
2. You saw two ways a capability-based restriction can be undermined: `--privileged` handing back everything, and Docker's port sysctl making the gate irrelevant. Generalise: why is "I dropped the dangerous capability" not the same as "the dangerous action is prevented"?
3. Capstone link. The capstone target reaches its container step through a container that keeps a capability it should not. You have not seen that container yet. From the capabilities you named in Exercise 3a, which single one would you check for first and drop, why, and how would you verify the drop by reading the container's sets?

References

- `capabilities(7)` (the full list and the five sets); `capsh(1)`, `getcap(8)`, `setcap(8)`.
- Docker default capability set: the `--cap-add`/`--cap-drop`/`--privileged` documentation.
- `ip_unprivileged_port_start` and the reserved-port change in Docker 20.10.
- W. Du, *Computer Security: A Hands-on Approach*, 3rd ed., 2022, the capabilities material.

Deliverables

There is **no separate report for this lab**; keep your capability transcripts. Block 3's assessed deliverable is **Lab report 3**, produced in Lab 8, which collects this lab's preparation questions P1-P2 and its Reflection. The least-privilege reasoning here (4a, and the "start from nothing and add" principle) is directly reused in Lab 11's compose-hardening report and in the capstone.